NLP MINI PROJECT SUMMARY REPORT

PolluAlert: An NLP-Based Pollution Emergency Detection and Alert System

1. Introduction

Pollution incidents such as chemical leaks, sewage discharge, waste burning and noise nuisance are usually first reported by citizens as short free-text complaints. Reading and sorting such text by hand is slow, and urgent cases can be delayed. PolluAlert is a Natural Language Processing (NLP) system that screens an incident description and returns the pollution type, severity, emergency status, location, key entities and keywords. It is a preliminary text-screening aid that complements, and does not replace, sensor-based monitoring and official inspection.

2. Problem Statement

Existing monitoring depends mainly on hardware sensors, while citizen complaints arrive as unstructured text and are handled manually, with no automatic categorisation, no severity or urgency flag, and no feedback loop. The problem is to automatically classify a free-text pollution report into a pollution category, a severity level and an emergency status, and to extract the useful details from it, so that serious cases can be noticed earlier.

3. Objectives

1. To build a labelled dataset of pollution incident descriptions.
2. To preprocess text using a negation-aware NLP pipeline and extract TF-IDF features.
3. To train and compare Naive Bayes, Logistic Regression and Linear SVM classifiers.
4. To classify incidents by pollution type, severity and emergency status.
5. To extract locations, entities and keywords and generate a structured incident summary.
6. To store results in an SQLite database and collect user feedback through a Streamlit interface.

4. Proposed Methodology

- **Dataset:** a student-curated set of 430 labelled incident records, split 80/20 (stratified) into 344 training and 86 test records.
- **Preprocessing:** lower-casing, cleaning, tokenisation, stop-word removal and lemmatisation, with negation handling so that phrases such as "no leak" are not treated as incidents.
- **Feature extraction:** TF-IDF with unigrams and bigrams (up to 5,000 features).
- **Models:** Multinomial Naive Bayes, Logistic Regression and Linear SVM, each trained for three tasks.
- **Pollution type (8 classes):** Air, Water, Soil/Land, Noise, Industrial, Waste/Garbage, Chemical and Other/Unknown.
- **Severity:** Low, Medium or High, supported by rule-based escalation for danger cues.
- **Emergency status:** Normal, Pollution Report or Potential Emergency.

- **Information extraction:** regular-expression patterns, with an optional spaCy fallback, for locations, organisations, pollution sources, water bodies and affected groups; keyword extraction and summary generation.
- **Workflow:** user text → preprocessing → TF-IDF → classifiers → rules and extraction → result display → SQLite storage → user feedback (correct / incorrect).

5. Technologies Used

Technology	Purpose
Python	Core programming language
NLTK	Tokenisation, stop words, lemmatisation
Scikit-learn	TF-IDF, ML models (NB, LR, SVM), evaluation
spaCy	Named entity recognition (with regex fallback)
Pandas, NumPy	Data loading and numerical operations
Matplotlib, Seaborn	Charts and visualisations
Joblib	Saving and loading trained models
Streamlit	Web user interface
SQLite	Storage of reports and feedback

6. System Implementation

PolluAlert is implemented as a Streamlit web application with three pages: Home, Pollution Analysis and Analytics Dashboard. The user types or selects an example incident and presses "Analyze Incident" (Fig. 1). The system then displays the emergency status, pollution type, severity, location, model confidence, keywords and extracted entities (Fig. 2). Every analysis is saved to SQLite, and the user can mark the result as correct or incorrect.

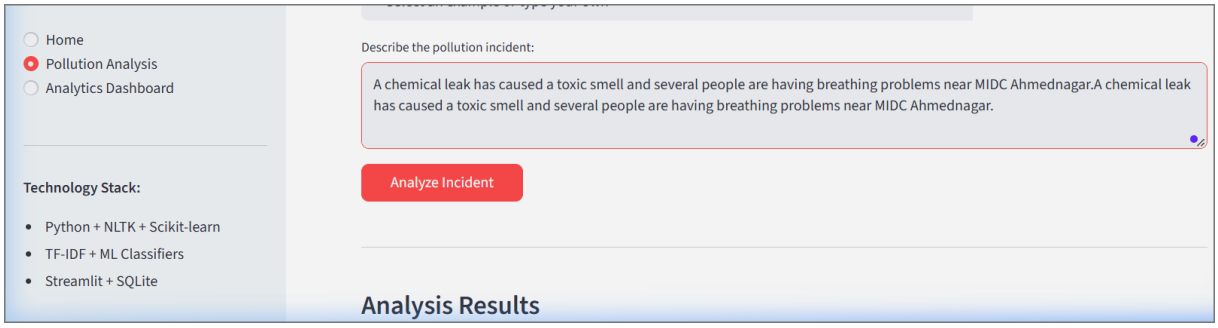


Fig. 1: Pollution Analysis page with incident input

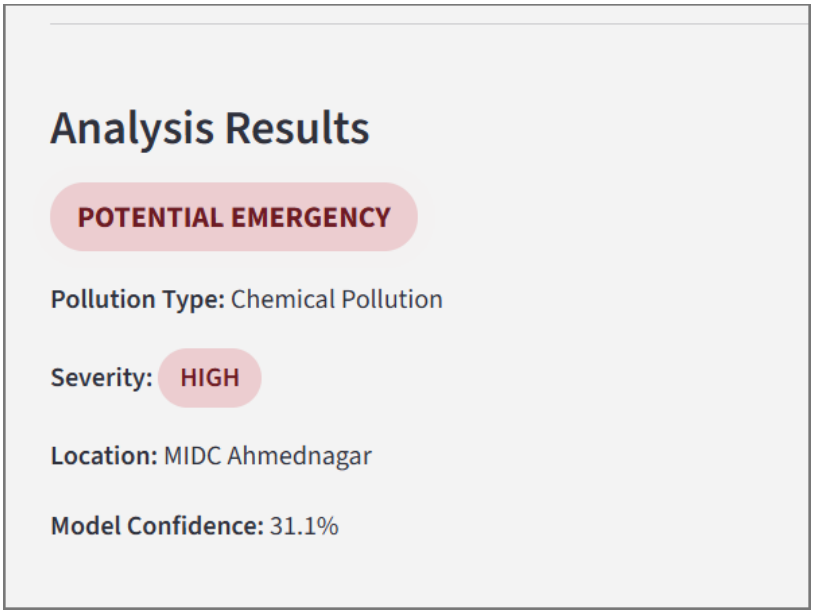


Fig. 2: Analysis results for a chemical-leak incident

Fig. 2 shows a Potential Emergency, Chemical Pollution, High severity, location MIDC Ahmednagar, with a model confidence of 31.1%, a low value that reflects the small dataset; such outputs are indicative only. The same page also lists the extracted keywords and entities and an incident summary.

7. Results and Evaluation

Experimental setup: 430 records, stratified 80/20 split (344 train / 86 test), random_state 42, TF-IDF (1,2)-grams with a maximum of 5,000 features, weighted-average metrics, Python 3.13 on Windows. The three models were compared on each task; weighted F1-scores are given in Table 1.

Table 1: Weighted F1-score of the models on the 86-record test set

Task	Naive Bayes	Logistic Regression	Linear SVM
Pollution type	0.5774	0.7184	0.6517
Severity	0.5861	0.6708	0.6316
Emergency status	0.7160	0.8929	0.8900

Logistic Regression gave the highest F1 on all three tasks, 0.7184 for pollution type, 0.6708 for severity and 0.8929 for emergency status. Linear SVM was close on emergency status (0.8900). Severity was the hardest task because Low, Medium and High descriptions overlap in wording. These are F1-scores, not accuracy, and they come from a small test set, so they should be read as indicative. The automated test suite (22 of 22 pytest tests) passed, and all 10 sample test cases passed.

8. Conclusion and Future Scope

PolluAlert shows that classical NLP, TF-IDF features with Logistic Regression, can turn free-text pollution complaints into structured information with a pollution type, severity, emergency status and extracted details, inside a simple web application with storage and feedback. The outputs are preliminary text-based assessments. They do not replace sensors, field inspections or expert verification. The main limits are the small dataset of 430 records and English-only input.

Future scope, as listed in the project report:

- Multilingual support for Hindi, Marathi and other Indian languages.
- A larger dataset of 2,000–5,000 records.
- Transformer-based models such as BERT or RoBERTa.
- Integration with IoT sensor data and geolocation mapping.
- Retraining on collected user feedback to improve the models over time.
- Connection to CPCB / SPCB complaint portals.